AIAssistedCoding

ASSIGNMENT2.3

Name: PRUDVI RAJ
HTNo:2303A52349
Batch:32

Question:

Task1:
WordFrequencyfromTextFile
Scenario:Youareanalyzinglogfilesforkeywordfrequency.
Task:UseGeminitogeneratePythoncodethatreadsatextfileand counts word frequency, then explains the code

Prompt:

CreateaPythonprogramthatcreatesatextfile,writessampletextintoit, then reads the file and displays its contents.
Addclearcommentsinthecode.

CodeScreenshot:

```
import re
from collections import Counter

def count_word_frequencies(filepath):
    """
    Reads a text file, counts the frequency of each word, and returns the counts.
    Words are converted to lowercase and punctuation is ignored.
    """
    try:
        with open(filepath, 'r') as file:
            text = file.read()
            # Convert text to lowercase and find all word sequences (alphanumeric)
            # \b matches word boundaries, \w+ matches one or more word characters
            words = re.findall(r'\b\w+\b', text.lower())

            # Use collections.Counter to easily count word occurrences
            word_counts = Counter(words)
            return word_counts
    except FileNotFoundError:
        return f"Error: The file '{filepath}' was not found. Please ensure it exists."
    except Exception as e:
        return f"An unexpected error occurred: {e}"

# --- Main execution ---
# Assuming the text file is named 'words.txt' and is located in the '/content/' directory.
# You can change 'file_path' if your file is named differently or in another location.
file_path = '/content/words.txt'

print(f"Reading file: {file_path}\n")

frequencies = count_word_frequencies(file_path)

# Check if the function returned a dictionary (Counter object) or an error string
if isinstance(frequencies, dict):
    print("Word Frequencies (most common first):")
    # Iterate through the most common words and their counts
    for word, count in frequencies.most_common():
        print(f"  '{word}': {count}")
else:
    print(frequencies) # Print the error message
```

Output(O/P):

```
print(frequencies) # Print the error mess

Reading file: /content/words.txt

Word Frequencies (most common first):
  'apple': 4
  'banana': 3
  'orange': 3
  'dog': 3
  'cat': 2
```

Explanation:

Thisprogramreadsatextfileandsplitsthecontentintowords.

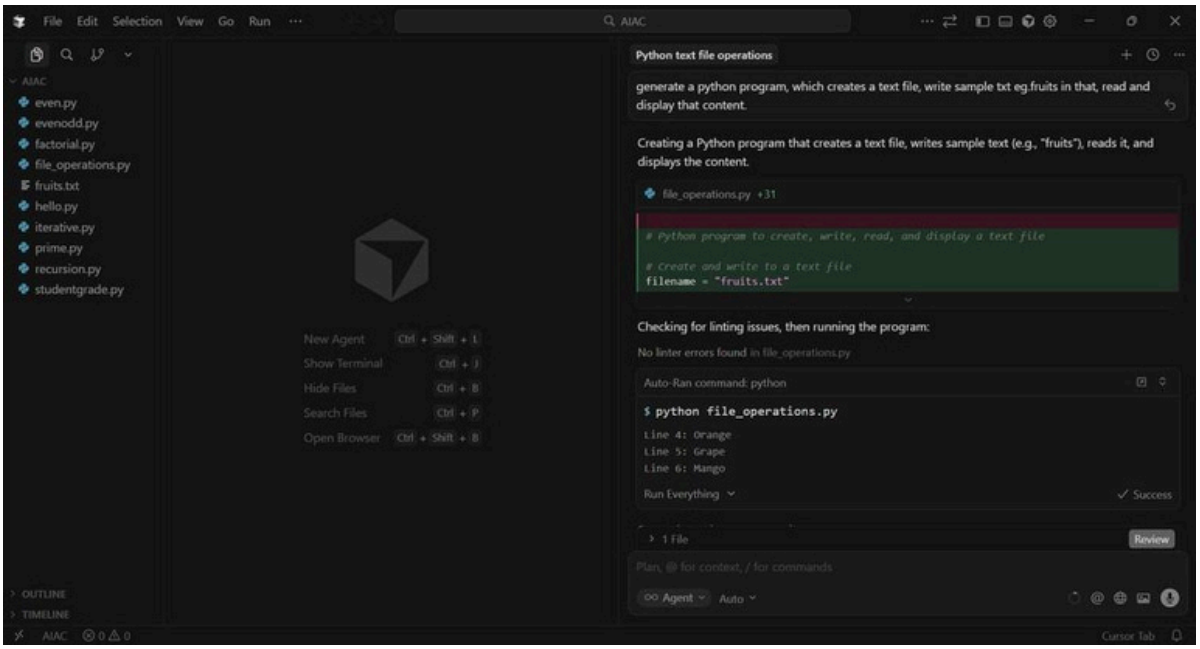
Eachwordisstoredinadictionarywherethekeyisthewordandthevalueisitscount. If a word appears again, its count is increased.
Finally,theprogramprintshowmanytimeseachwordappearsinthefile. This helps in analyzing text data and finding frequently used words.

Question:

Ta sk 2:
FileOperationsUsingCursorAI
Scenario:Youareautomatingbasicfileoperations.
Task: Use Cursor AI to generate a program that:
Creates a text file
Writessampletext
Readsanddisplaysthecontent

Prompt:

generateapythonprogram,whichcreatesatextfile,writesampletxteg.fruitsinthat,read and display that content.



CodeScreenshot:

file_operations.py

file_operations.py > ...

```python
# Python program to create, write, read, and display a text file
# Create and write to a text file
filename = "fruits.txt"

# Write sample text to the file
with open(filename, 'w') as file:
    file.write("fruits\n")
    file.write("Apple\n")
    file.write("Banana\n")
    file.write("Orange\n")
    file.write("Grape\n")
    file.write("Mango\n")

print(f"File '{filename}' has been created and written to.\n")

# Read and display the content
print("Reading and displaying the content:")
print("-" * 40)
with open(filename, 'r') as file:
    content = file.read()
    print(content)
print("-" * 40)

# Alternative: Read line by line
print("\nReading line by line:")
print("-" * 40)
with open(filename, 'r') as file:
    for line_num, line in enumerate[str](file, 1):
        print(f"Line {line_num}: {line.strip()}")
```

Output(O/P):

```
Problems   Output   Debug Console   Terminal   Ports

$ python file_operations.py
File 'fruits.txt' has been created and written to.

Reading and displaying the content:
----------------------------------------
fruits
Apple
Banana
Orange
Grape
Mango

----------------------------------------

Reading line by line:
----------------------------------------
Line 1: fruits
Line 2: Apple
Line 3: Banana
Line 4: Orange
Line 5: Grape
Line 6: Mango
$ 
```

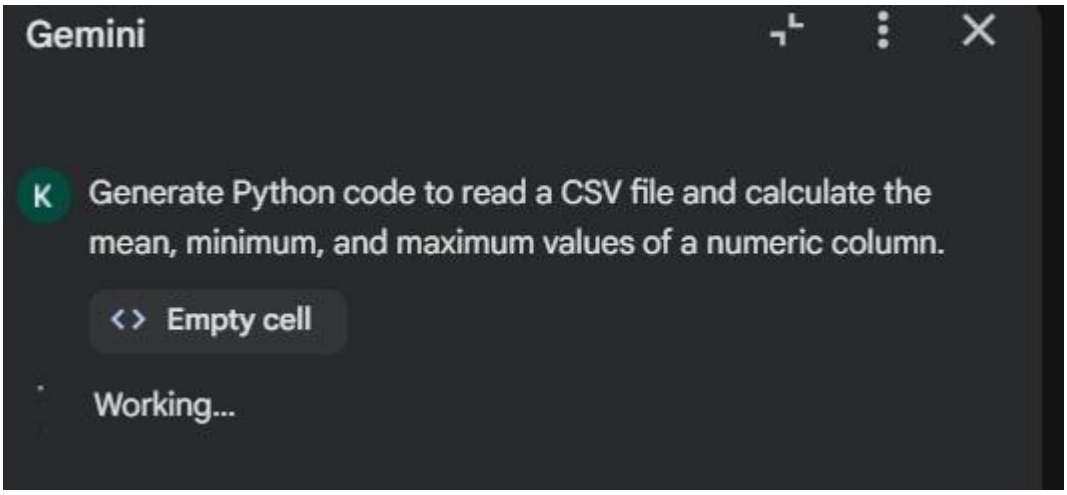
Explanation:

ThisprogramdemonstratesbasicfilehandlinginPythonusingCursor.

First,atextfileiscreatedandsampletextiswrittenintoit.
Then,thesamefileisopenedinreadmodeanditscontentsaredisplayedonthescreen. It shows how Python can be used to create, write, and read files easily.
Suchoperationsareusefulinautomationanddatastoragetasks.

Question:
Task3:CSVDataAnalysis
Scenario:YouareprocessingstructureddatafromaCSVfile.
Task:UseGeminiinColabtoreadaCSVfileandcalculatemean,min, and max.

Prompt:
GeneratePythoncodetoreadaCSVfileandcalculatethemean, minimum,and maximum values of a numeric column.



CodeScreenshot:

```python
import pandas as pd
import io

# Uncomment the lines below and replace 'your_file.csv' with your CSV file path
try:
    df = pd.read_csv('scores.csv')
except FileNotFoundError:
    print("Error: CSV file not found. Please check the path.")
    exit()

# Specify the numeric column you want to analyze
column_name = 'score'

# Check if the column exists in the DataFrame
if column_name in df.columns:
    # Ensure the column is numeric (e.g., float or int)
    # pd.to_numeric will convert values to numeric, coercing errors to NaN
    numeric_column = pd.to_numeric(df[column_name], errors='coerce')

    # Drop rows where the numeric conversion resulted in NaN (non-numeric values)
    numeric_column = numeric_column.dropna()

    if not numeric_column.empty:
        # Calculate mean, minimum, and maximum
        mean_value = numeric_column.mean()
        min_value = numeric_column.min()
        max_value = numeric_column.max()

        print(f"Statistics for column '{column_name}':")
        print(f"  Mean: {mean_value:.2f}")
        print(f"  Minimum: {min_value:.2f}")
        print(f"  Maximum: {max_value:.2f}")
    else:
        print(f"Column '{column_name}' contains no valid numeric data after cleaning.")
else:
    print(f"Error: Column '{column_name}' not found in the CSV file. Available columns: {df.columns.tolist()}")
```

Output(O/P):

```
•••  Statistics for column 'score':
       Mean: 82.00
       Minimum: 67.00
       Maximum: 92.00
```

Explanation:

ThisprogramreadsdatafromaCSVfileusingPython.
Itextractsnumericalvaluesfromacolumnandcalculatesthemean, minimum, and maximum.
CSVanalysisisusedindataprocessingandanalyticsapplications.

Question:

Task4:SortingListsManualvsBuilt-in
Scenario:Youarereviewingalgorithmchoicesforefficiency. Task:
Use Gemini to generate:
Bubblesort
Python’sbuilt-insort()
Comparebothimplementations.

Prompt:

K Generate Python code for Bubble Sort

Empty cell

Here's the Python code for implementing the Bubble Sort algorithm. After the code, I'll explain how it works in simple terms.

Sources

1. github.com/rbernalop/codemmunity

K Generate Python code for built-in sort

Certainly! Python provides excellent built-in ways to sort lists. Here are examples using both the `list.sort()` method and the `sorted()` function, along with an explanation of their differences.

CodeScreenshot:

ManualBubbleSortCode:

```python
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place
        for j in range(0, n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j] # Swap elements
    return arr

# --- Example Usage ---
my_list = [64, 34, 25, 12, 22, 11, 90]
print("Original list:", my_list)

sorted_list = bubble_sort(my_list)
print("Sorted list using Bubble Sort:", sorted_list)

my_list_2 = [5, 1, 4, 2, 8]
print("\nOriginal list 2:", my_list_2)

sorted_list_2 = bubble_sort(my_list_2)
print("Sorted list 2 using Bubble Sort:", sorted_list_2)
```

Sortingusingsort()function:

```python
# --- Using the list.sort() method ---
# This method sorts the list in-place (modifies the original list) and returns None.

my_list_1 = [64, 34, 25, 12, 22, 11, 90]
print("Original list (list.sort()):", my_list_1)

my_list_1.sort()
print("Sorted list (list.sort()):", my_list_1)

# You can also sort in descending order
my_list_2 = ['banana', 'apple', 'cherry', 'date']
print("\nOriginal list (descending):", my_list_2)

my_list_2.sort(reverse=True)
print("Sorted list (descending):", my_list_2)
```

Output(O/P):

Bubblesort:

```
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list using Bubble Sort: [11, 12, 22, 25, 34, 64, 90]

Original list 2: [5, 1, 4, 2, 8]
Sorted list 2 using Bubble Sort: [1, 2, 4, 5, 8]
```

SortFunction:

```
Original list (list.sort()): [64, 34, 25, 12, 22, 11, 90]
Sorted list (list.sort()): [11, 12, 22, 25, 34, 64, 90]

Original list (descending): ['banana', 'apple', 'cherry', 'date']
Sorted list (descending): ['date', 'cherry', 'banana', 'apple']
```

Explanation:

Bubble sort is a simple sorting algorithm that repeatedly compares and swaps adjacent elements. It is easy to understand but inefficient for large data sets. Python's built-in sort functionisshorter,optimizedandmuchfaster.Thebuilt-inmethodshouldbepreferredin real-world applications.